

Where Does Streaming State Cost Go? A Reproducible Comparison of Flink and Kafka Streams on Kafka

Kiran N Kumar
knkumar@iu.edu

Santosh K Saminathan
s13.santosh@gmail.com

Draft checked against repository state as of 2026-07-21T09:26:07-07:00.

Abstract

Flink and Kafka Streams both keep local state in RocksDB and both offer exactly-once processing over Kafka, but they place durability in different system components: Flink coordinates asynchronous checkpoints from a JobManager, while Kafka Streams replicates state changes to broker-held changelog topics. This paper builds a benchmark with five workloads spanning stateless transforms, tumbling and sliding windows, and a stream-stream join, run on Apache Kafka 4.3.1 (KRaft mode), Kafka Streams 4.3.1, and Flink 2.2.0. It reports fixed-load visibility-delay measurements, worker and broker container samples, configuration sensitivity, and fault-injection outcomes on one shared host. Fifty independent correctness trials, five per engine/workload combination, matched their expected output with zero missing, unexpected, or duplicate records. Under 100 events/sec load for 30 minutes, the combined extttT2-T1 hop ranges from 4-6 milliseconds for W1-W2 to 4.3-6.2 seconds for W3-W4; it includes both event-time waiting and engine work, because the harness does not yet record semantic eligibility extttT_e. A sustained W3 tuning study contains three valid trials for each engine and interval, each with 6,000 inputs and 997 matched outputs. Fault injection shows that several Kafka Streams stateful trials did not finish within the observation window while the corresponding Flink trials completed, but the artifact does not directly measure recovery milestones or prove permanent data loss.

1. Introduction

A team choosing between Flink and Kafka Streams is choosing between two different places to put the cost of fault tolerance. Flink runs as its own cluster and drives recovery from checkpoints written to a configured checkpoint store. Kafka Streams runs inside the application’s own JVM and drives recovery from Kafka-broker-held changelog topics, so its blast radius and its dependency footprint

are different even when the two systems are asked to do the same job with the same guarantees.

Public comparisons of the two usually report a single number, records per second per core, and move on. That number hides where the cost actually sits: in the broker, in the checkpoint store, in the worker’s own commit protocol, or in how long a consumer group takes to rebalance after a crash. A team picking an engine for a workload with large keyed state, frequent restarts, or a hard latency SLA needs to know which of those costs it is signing up for, not just the steady-state throughput of a demo workload.

This paper reports on a benchmark built to make those costs visible rather than assume them. It instruments Kafka Streams and Flink identically: the same Kafka cluster, the same input generator, the same multiset verifier, and a four-point timestamp decomposition (T0 event generation, T1 broker ingestion, T2 engine output, T3 downstream visibility) computed the same way for both engines. The workload suite runs from a stateless identity transform up to a windowed stream-stream join, so a single-workload conclusion is structurally impossible. Section 5 reports what that harness has actually measured so far: correctness across all five workloads, latency under light and sustained load, a first fault-injection pass, and the shape of a stateful-window cost that both engines pay, not a Kafka-Streams-specific tax, though each engine gates that cost through a mechanically different part of its own pipeline.

2. Related Work and Background

Stream-processing benchmarks are an established genre, and this paper positions itself against them rather than beside them. The Nexmark benchmark defined a suite of continuous queries (filters, aggregations, joins) over an online-auction data model and is still the template most engine-specific query benchmarks follow.[9] The Yahoo Streaming Benchmark measured Storm, Flink, and Spark Streaming on a single windowed ad-count job and reported end-to-end latency and sustainable throughput as its headline numbers.[10] ESPBench built an enterprise-oriented suite over Kafka and ran it on Spark, Flink, and Hazelcast Jet, reporting query-result correctness and latency.[11] Theodolite framed the question as scalability, measuring how far Flink, Kafka Streams, and Beam-backed engines scale for a fixed load per instance.[12] DSPBench collected fifteen application workloads across domains and characterized them by selectivity, processing cost, and memory footprint.[13] What these share is that latency and throughput are reported as single aggregate numbers per system, and the comparison target is usually raw performance under load. This paper differs on two axes rather than on workload novelty. First, it decomposes each latency number into a four-point T0-T3 breakdown so a slow total can be attributed to broker ingestion, engine processing, or commit-and-consume visibility rather than left as one figure. Second, it treats fault-recovery cost as a first-class measured quantity for both engines rather than a steady-state footnote. The narrow engine pair, Flink and Kafka Streams, is deliberate: they are the two

systems that keep local RocksDB state and offer exactly-once over Kafka but place durability in different tiers, which is what makes a hop-level decomposition comparable across the two.

Apache Flink’s checkpointing documentation describes checkpoints as the mechanism that lets a restarted job resume as though failure had not happened, coordinated by asynchronous barriers flowing through the dataflow graph and landing in a configured durable store.[1,2] Apache Kafka Streams documents internal changelog topics as the recovery path for state stores: every local RocksDB write is also sent to a compacted Kafka topic, and a restarted instance rebuilds its store by replaying that topic before rejoining processing.[3,4] Apache Kafka’s KRaft mode replaces the ZooKeeper-based controller with a Raft-based quorum embedded in the brokers themselves.[5]

These are the two recovery paths this paper’s fault-injection experiments (Section 5.6) are designed to exercise: a Flink JVM or node kill should recover from a checkpoint, and a Kafka Streams JVM or node kill should recover by replaying a changelog. The proposal that preceded this paper (`proposal.md`) targets the EDBT Experiments, Analysis, and Benchmarks track specifically because it found the worker-throughput-only framing of the benchmarks above insufficiently informative for infrastructure cost decisions. This paper’s contribution is the decomposed-latency and fault-cost harness and its first round of measurements, not a new query suite or a survey of the field.

3. Research Questions

1. **RQ1, fixed-load latency and correctness.** At 100 events/sec on the documented shared host, how do downstream visibility delay, lag observations, and externally visible correctness differ across stateless and windowed workloads?
2. **RQ2, preliminary container resource observations.** What worker and broker CPU, memory, network-I/O, and block-I/O observations does the harness record under the evaluated load?
3. **RQ3, recovery and reprocessing behavior.** How do checkpoint or commit cadence and local-state loss affect recovery time, backlog drain, duplicate records, and downstream visibility delay?

This paper does not estimate a saturation boundary, total-system cost distribution, or recovery time. RQ1 has five independent correctness trials for each engine/workload combination and a fixed-rate latency picture for W1-W4. RQ2 reports container samples, not normalized infrastructure cost. RQ3 reports post-injection output observations and final verification within a bounded observation window.

4. Methodology

4.1 Workloads

Five workloads isolate where architectural divergence could matter: **W1** identity (one output per input, no state), **W2** filter/map (deterministic parity filter and doubling, no state), **W3** tumbling count (60-second event-time windows, keyed count), **W4** sliding sum (10-minute window, 1-minute slide, keyed sum), and **W5** stream-stream join (two input streams, 10-minute join window). **W3** and **W4** emit final-only output after the window closes rather than continuously updating intermediate results, so both engines are compared on the same output contract. Every workload uses a deterministic generator (100 keys, seed 7) and a shared multiset verifier that reports missing, unexpected, and duplicate output records rather than relying on ordering, since event-time correctness cannot be checked by output order alone.

4.2 Measurement harness

Every input record carries a deterministic ID. A load-generation process records **T0** immediately before writing it to Kafka. Kafka’s `LogAppendTime` on the input topic gives **T1** (broker ingestion). The engine stamps a wall-clock timestamp into the output payload immediately after its business logic runs, giving **T2** (engine output, pre-commit). A `read_committed` consumer records **T3** when it observes the committed output. The harness reports **T1-T0**, **T2-T1**, and **T3-T2**. For **W3** and **W4**, **T2-T1** combines semantic event-time waiting, triggering, and engine computation. It is not an engine-only processing metric because no watermark or stream-time eligibility timestamp (**T_e**) is recorded. The results therefore describe a measured combined hop, not a causal decomposition of engine work.

Both engines run against the same `apache/kafka:4.3.1` broker brought up in KRaft mode following the Apache Kafka quickstart [6], write output with `read_committed-visible` exactly-once delivery (Kafka Streams `exactly_once_v2`, Flink `EXACTLY_ONCE` Kafka sink), and are verified by the same external JSONL verifier. Flink 2.2.0 with `flink-connector-kafka` 5.0.0-2.2 [8] replaced an originally proposed Flink 2.3.x target after Apache Flink’s downloads page [7] showed 5.0.0 as compatible with Flink 2.1.x and 2.2.x, not 2.3.x. All experiments share one host, Docker daemon, physical CPU, and disk; the current artifact does not establish dedicated-host isolation or matched resource limits.

4.3 What has and has not been run

The artifact includes a randomized five-trial correctness runner, a fixed-rate latency runner, a 30-minute 100 events/sec stability runner, container resource sampling, a three-trial-per-cell **W3** tuning runner, and a five-trial fault-injection runner. Saturation, partition, parallelism, and state-size sweep scripts exist, but their result matrices are incomplete and are not used for claims in this paper.

5. Results

5.1 Correctness

Five independently ordered correctness trials per engine/workload combination passed against the shared deterministic input. All 50 runs matched their expected outputs exactly: zero missing, zero unexpected, and zero duplicate records, across output counts ranging from 199 (W3) to 11,024 (W5). The earlier baseline and `repeat1` runs are retained in the artifact but are not the basis for this count. This establishes only finite-input functional correctness under `read_committed` delivery; it does not establish correctness near saturation or after an unbounded recovery period.

5.2 Latency under light, fixed-rate load

At 20 records/sec with 100 input records, both engines pass every workload's correctness check while p99 end-to-end visibility delay ranges from about 2.6 seconds (W1, both engines) up to about 7.1 seconds (Kafka Streams W4) and 5.8 seconds (Flink W4). Kafka Streams and Flink each ran a bounded 10/20/40 records/sec sweep plus a second 20/sec run. The windowed workloads (W3, W4) carry roughly two to three times the p99 of the stateless workloads. The rate direction is inconsistent for the windowed workloads: Kafka Streams W3 falls from 11,631 ms at 10/sec to 3,943 ms at 40/sec. With only 100 inputs, production duration changes the position of the final tick relative to window boundaries; this is a finite-input artifact, not an estimate of sustainable throughput. Section 5.3 uses 1,800 times as many input records and is the stronger fixed-load evidence.

5.3 Stability and latency decomposition under sustained load

Both engines ran a 100 events/sec, 30-minute (180,000-event) open-loop stability check on W1 through W4. Backlog did not grow monotonically in any of the eight runs (`experiments/results/*_latency_stability_100/lag.csv`), and all eight passed correctness verification exactly (180,000, 89,906, 29,924, and 30,900 matched records respectively, zero missing, unexpected, or duplicate in every case). The decomposed p99 latency is where the two engines, and the four workloads, separate:

The ingestion hop (T1-T0) is nearly identical across both engines and all four workloads, around one second. That behavior is consistent with producer-side pacing, but the harness does not isolate producer batching from broker work. The combined T2-T1 hop is 4-6 milliseconds for W1-W2 and 4.3-6.2 seconds for W3-W4. It includes the time that final-only window output waits for event-time progress, so it does not show that Kafka Streams spends 23-42% more engine computation than Flink. It shows only that the combined hop is larger in these single 30-minute runs. A T_e implementation and repeated, blocked stability trials are required before making a processing-cost comparison.

Getting to this table required fixing two bugs specific to the Flink side, doc-

Engine	Workload	p99 total	p99 t1-t0 (ingestion)	p99 t2-t1 (processing)	p99 t3-t2 (commit + consume)
Kafka Streams	W1 identity	1967.4 ms	1000.7 ms	6.0 ms	1008.6 ms
Kafka Streams	W2 filter_map	1928.3 ms	1000.5 ms	5.0 ms	1009.3 ms
Kafka Streams	W3 tumbling_count	6576.4 ms	991.9 ms	6083.0 ms	19.0 ms
Kafka Streams	W4 sliding_sum	6813.6 ms	999.2 ms	6249.0 ms	65.4 ms
Flink	W1 identity	1882.1 ms	1000.3 ms	4.0 ms	1001.0 ms
Flink	W2 filter_map	1874.9 ms	999.9 ms	4.0 ms	997.0 ms
Flink	W3 tumbling_count	5506.7 ms	991.7 ms	4287.0 ms	968.6 ms
Flink	W4 sliding_sum	5751.4 ms	1000.4 ms	5080.0 ms	994.9 ms

umented in full in `docs/project_log.md` (2026-07-17): a host-bind-mounted checkpoint directory that was never cleared between unrelated runs, which made a job try to resume an incompatible checkpoint left by a different workload and crash at startup; and a Kafka-consumer idle timeout hardcoded well below what a 30-minute run with no output until its final tick needs. W3 and W4 failed outright (zero output matched) on the first attempt for exactly these reasons, not because of anything about window semantics; the corrected runs above are what is reported everywhere else in this paper.

W5 (stream-stream join) is not run at the 180,000-event, 30-minute scale used for W1 through W4. Section 7 explains why and what was run instead.

5.4 Resource utilization

A `docker stats` sampler (`src/stream_state_bench/resource_monitor.py`) records CPU percentage, memory, cumulative network I/O, and cumulative block I/O for the engine and broker containers. The collected CSV files cover Kafka Streams W1-W4 and Flink’s stability sweep. Kafka Streams’ worker memory rises from a mean 117.5 MiB on W1 to 157.0 MiB on W4. Flink’s worker has a blended mean of 377.6 MiB across its stability sweep. These are shared-host container observations, not normalized CPU-seconds, disk-throughput measurements, checkpoint-storage accounting, or total-system cost.

5.5 Configuration sensitivity

Both engines expose their commit or checkpoint interval through `COMMIT_INTERVAL_MS` and `CHECKPOINT_INTERVAL_MS`. The tuning study uses W3 at 100 inputs/sec

for 60 seconds (6,000 inputs plus one tick). Each of the 12 trials produced and verified 997 final-window outputs. The table reports the median of three per-run nearest-rank p99 values for each engine/interval cell; its component values are not additive p99 decompositions.

Engine	Interval	p99 total	p99 t1-t0	p99 t2-t1 (processing)	p99 t3-t2 (commit)
flink	1000 ms	5596.7 ms	1002.0 ms	4259.0 ms	732.1 ms
flink	10000 ms	12643.2 ms	1001.8 ms	4240.0 ms	8702.2 ms
kafka_streams	1000 ms	6767.1 ms	1001.8 ms	6085.0 ms	93.7 ms
kafka_streams	10000 ms	23908.1 ms	1001.4 ms	23155.0 ms	137.2 ms

All 12 trials matched 997/997 expected outputs with zero missing, unexpected, or duplicate records. The median per-run p99 T2-T1 rises from 6,085 to 23,155 ms for Kafka Streams when the interval changes from 1,000 to 10,000 ms, while the corresponding T3-T2 median changes from 93.7 to 137.2 ms. For Flink, the median per-run p99 T2-T1 remains near 4.25 seconds and T3-T2 rises from 732.1 to 8,702.2 ms. These observations are compatible with different timing paths, but T2-T1 still includes semantic waiting and the experiment does not independently vary Kafka Streams cache settings. The study supports configuration sensitivity, not a complete causal explanation.

5.6 Post-injection output observations

`scripts/run-failure-test.sh` injects JVM kill, broker kill, and container recreation after local-state loss into W1, W3, and W4. The runner also contains KRaft, pause, and changelog-removal scenarios, but those are engine-specific or incomplete and are not used for the comparative table. `jvm_kill` is a process kill, not an OOM experiment. `node_loss` in the archived result names means container and local-volume loss, not removal of a physical host node.

On W1, all reported trials matched their expected output. For W3 and W4, several Kafka Streams JVM-kill and container/local-volume-loss trials did not complete within the runner’s observation window; those are censored harness outcomes, not evidence of permanent data loss. The corresponding reported Flink trials completed and passed final output verification. The harness records output timestamps and final verification, but not failure time, task assignment, restore start/completion, first post-recovery record, or backlog-zero time. It therefore cannot measure recovery duration or identify the mechanism behind an unfinished Kafka Streams trial.

Table 5.6 presents median per-run p99 T2-T1 values across the five trials. These values are post-injection output-delay summaries, not recovery times. DNF means that fewer than half of the trials completed within the observation window.

An earlier Kafka Streams W1 container-loss run returned 706 of 2,000 expected

Engine	Failure mode	W1 p99 T2-T1	W3 p99 T2-T1	W4 p99 T2-T1
flink	jvm_kill	11.07s (5/5)	22.86s (5/5)	22.18s (5/5)
flink	broker_kill	2.44s (5/5)	22.79s (5/5)	21.73s (5/5)
flink	node_loss	11.33s (5/5)	22.82s (5/5)	22.01s (5/5)
kafka_streams	jvm_kill	44.66s (5/5)	DNF (0/5)	DNF (0/5)
kafka_streams	broker_kill	2.56s (5/5)	26.73s (5/5)	23.51s (5/5)
kafka_streams	node_loss	44.05s (5/5)	DNF (1/5)	DNF (1/5)

outputs because the recreated container joined a new application identity. The runner now exports the original application identity before recreation; the corrected W1 trials in the table passed final verification. This incident shows why a completion timeout alone cannot establish an engine recovery failure.

6. Discussion

Final-only window output has a multi-second combined visibility hop in these runs. Section 5.3 shows T2-T1 rising from single-digit milliseconds on W1-W2 to 4.3-6.2 seconds on W3-W4. Kafka Streams’ suppression operator and Flink’s window trigger are different implementations, but the current metric does not separate event-time waiting from computation. The data support a workload-level observation about final-only window visibility under this configuration. They do not isolate the cost of either engine’s window operator.

The sustained tuning trials show that the two interval changes alter different measured hops. Kafka Streams’ median per-run p99 T2-T1 rises from 6,085 to 23,155 ms when COMMIT_INTERVAL_MS changes from 1,000 to 10,000 ms. Flink’s median per-run p99 T3-T2 rises from 732.1 to 8,702.2 ms when CHECKPOINT_INTERVAL_MS changes over the same range. This pattern motivates instrumentation of suppression release, stream-time, cache flushes, checkpoints, precommits, and transaction commits. It does not establish those mechanisms from the present timestamps alone.

Broker-kill trials affect both pipelines. The W1 broker-kill trials show increased output delay in both engines. Because both systems share the same broker deployment and host, those observations do not isolate an engine-specific broker-failure cost.

Stateful fault outcomes require direct recovery instrumentation. Kafka Streams has several censored W3/W4 JVM-kill and container/local-volume-loss trials, while reported Flink trials complete and pass final verification. The current trace cannot distinguish delayed restoration, rebalance failure, transaction retry, an identity error, or a permanent correctness failure. It also cannot establish that Flink’s behavior is independent of broker health or that a Kafka Streams timeout is data loss. Those are open measurements, not architectural conclusions.

7. Threats to Validity

Single host, shared Docker daemon. Every container in this benchmark, both engines’ workers and the Kafka broker, runs on one machine sharing one Docker daemon and one set of physical CPU and disk resources with whatever else is running on that host. Resource-utilization numbers (Section 5.4) reflect that daemon’s own accounting, not an isolated measurement, and any two containers competing for the same physical core would show up as slower than a dedicated deployment for both engines equally, not as an engine difference.

Combined semantic waiting and computation. The latency probe lacks a semantic eligibility timestamp. T2-T1 is therefore a combined event-time-waiting and engine-output hop for W3-W5. It cannot support a claim about engine computation alone or a causal comparison of Kafka Streams and Flink processing cost.

No performance envelope. The repository has partial saturation and parallelism scripts, but not a complete matrix with repeated trials, a bounded-latency definition, lag-slope uncertainty, or correctness verification at every rate. The paper is limited to fixed-load observations.

The join workload’s output cardinality does not scale linearly with input volume. `stream_stream_join`’s expected-output count depends on how many left and right events fall within each other’s 10-minute join window, not just on the input count. At 1,000 left and 1,000 right events, the reference generator already produces 11,024 expected outputs. Applying the same 180,000-event, 30-minute stability parameters used for W1 through W4 to W5 produced a 3.2 GB expected-output file and a container that ran past 11 hours without the correctness probe completing. We treat this as an out-of-scope saturation failure for W5 under these parameters. (see `docker-compose.log` for the `w5_latency_stability` incomplete run). W5’s stability figures in this paper use a bounded 120-second, 12,000-event run instead. This is a benchmark-design finding worth stating on its own: an open-loop stability protocol tuned for workloads with output volume close to input volume is not safe to apply unmodified to a join workload without first bounding the join window relative to the intended run duration, or sampling rather than fully verifying expected output.

Version substitution was compatibility-driven. Flink 2.2.0 replaced an originally proposed Flink 2.3.x because the Kafka connector version this benchmark needs documents compatibility with 2.1.x and 2.2.x, not 2.3.x (Section 4.2). This is disclosed rather than silently absorbed, but it does mean any reader comparing this paper’s Flink numbers against a 2.3.x deployment is comparing across a version this paper did not test.

8. Conclusion and Future Work

Fifty independent finite-input correctness trials passed across the two engines and five workloads. At the evaluated 100 events/sec fixed load, the measured combined T2-T1 hop is multi-second for W3 and W4 while it remains single-digit milliseconds for W1 and W2. The sustained W3 tuning study contains three valid trials per engine/interval cell and shows that changing the two interval settings changes different measured hops. The failure harness provides useful final-verification outcomes but does not measure recovery duration or identify the cause of a censored run.

The next revision must add T_e with raw watermark or stream-time evidence, use per-run rather than pooled inference, record direct recovery milestones and state-restoration measurements, and either complete the saturation and total-cost studies or retain the fixed-load, container-observation scope used here.

References

1. Apache Flink, “Checkpointing”: <https://nightlies.apache.org/flink/flink-docs-stable/docs/dev/datastream/fault-tolerance/checkpointing/>
2. Apache Flink, “Fault Tolerance”: https://nightlies.apache.org/flink/flink-docs-stable/docs/learn-flink/fault_tolerance/
3. Apache Kafka, “Managing Streams Application Topics”: <https://kafka.apache.org/40/streams/developer-guide/manage-topics/>
4. Apache Kafka, “Architecture”: <https://kafka.apache.org/31/streams/architecture/>
5. Apache Kafka, “KRaft”: <https://kafka.apache.org/35/operations/kraft/>
6. Apache Kafka, “Quickstart”: <https://kafka.apache.org/quickstart/>
7. Apache Flink, “Downloads”: <https://flink.apache.org/downloads/>
8. Maven Central, `org.apache.flink:flink-connector-kafka`: <https://central.sonatype.com/artifact/org.apache.flink/flink-connector-kafka>
9. P. Tucker, K. Tufte, V. Papadimos, and D. Maier, “NEXMark: A Benchmark for Queries over Data Streams (Draft),” OGI School of Science and Engineering, Oregon Health and Science University, technical report, 2002: <https://datalab.cs.pdx.edu/niagara/pstream/nexmark.pdf>
10. S. Chintapalli, D. Dagit, B. Evans, R. Farivar, T. Graves, M. Holderbaugh, Z. Liu, K. Nusbaum, K. Patil, B. J. Peng, and P. Poulosky, “Benchmarking Streaming Computation Engines: Storm, Flink and Spark Streaming,” in Proc. IEEE International Parallel and Distributed Processing Symposium Workshops (IPDPSW), 2016, pp. 1789-1792. doi:10.1109/IPDPSW.2016.138
11. G. Hesse, C. Matthies, M. Perscheid, M. Uflacker, and H. Plattner, “ESPBech: The Enterprise Stream Processing Benchmark,” in Proc. ACM/SPEC International Conference on Performance Engineering (ICPE), 2021, pp. 201-212. doi:10.1145/3427921.3450242
12. S. Henning and W. Hasselbring, “Theodolite: Scalability Benchmarking of Distributed Stream Processing Engines in Microservice Architectures,” Big

- Data Research, vol. 25, article 100209, 2021. doi:10.1016/j.bdr.2021.100209
13. M. V. Bordin, D. Griebler, G. Mencagli, C. F. R. Geyer, and L. G. L. Fernandes, “DSPBench: A Suite of Benchmark Applications for Distributed Data Stream Processing Systems,” *IEEE Access*, vol. 8, pp. 222900-222917, 2020. doi:10.1109/ACCESS.2020.3043948